



# PHP loops 75m

Control structures are used to alter how the code is executed using conditional statements and loop statements. Generally, code is executed sequentially and line by line, but control structures change this by allowing the script to respond differently depending on different inputs and situations.

For example, PHP can be used to check whether a user is logged into a website. Based on the user's login status, they'll either be redirected to a login page or be able to progress to their account details page. This action is done through control structures that respond differently (and execute the code differently) depending on whether the user is logged in or not.

PHP has control structures to provide for iteration and selection. In this section, you will begin by looking at some syntax options for different types of iteration (**for**, **while** and **foreach** loops) and selection (**if** and **switch** statements).

## for loop statements

The PHP **for** loop statement loops through (executes) one or more statements in a code block a fixed number of times based on changing the value of a variable each time it goes through the loop. The **for** loop is generally used when you know how many times the script should be run.

The syntax is as follows:

```
for ([counter start value]; [condition to end looping]; [counter update])  
{  
    // code to be executed  
}
```

Skip to main content

```
{  
  
//loop over something  
  
}
```

All three parts of the loop are optional, provided you retain the semicolon (;) that separates each part.

The following example loops over the statements 10 times.

```
for ($i = 0; $i < 10; $i++ )  
{  
  
// displays the numbers 0 to 9, separated by spaces  
  
echo $i. " ";  
  
}
```

Notice the variable initialisation (as in `$i` above) is part of the **for** loop. It is a generally accepted practice in most programming languages with **for** loops to use `$i`, `$j` and `$k` as the variable names for the loop counter. You generally don't want to have loops four levels deep, which is why there are only three variable names.

Execute this code in the JDoodle example below.



ENTER FULLSCREEN

[Skip to main content](#)

# The simplest **coding platform**



Skip to ☒ main content

## Writing a for loop

Non-Assessed

1. Write a for loop that will print the following sequence of numbers:  
■ 30 25 20 15 10 5 0
2. Test out your code to see whether your loop statement produces this sequence of numbers.

☐ I successfully executed this code

Submit



## 'while' loop statements

The PHP **'while'** statement will continually execute statements while the condition is true. This statement requires the test expression of the condition to be set **before** entering the loop. Then, while in the loop, the variable or expression that controls the loop needs to be updated.

This loop is similar to the **'do...while'** loop, except 'do...while' will always execute the loop contents once and then continue to execute the statement while the condition is true.

The syntax used for the **'while'** loop statement is:

```
while (expression)
```

```
{
```

```
// code to execute as long as expression evaluates to TRUE
```

Skip to main content

```
| }
```

Execute the code in the JDoodle example below to see the numbers 0 to 9 displayed in the output.



ENTER FULLSCREEN

---

[Skip to main content](#)

## The simplest **coding platform**



[Skip to main content](#)

## 'do...while' loop statement

The syntax used for the '**do...while**' loop is:

```
do {  
  
    code to be executed;  
  
} while (condition is true);
```

The following example uses the '**do...while**' loop statement to display the numbers from 0 to 9 by executing the code until \$i is 10 or larger.



ENTER FULLSCREEN

[Skip to main content](#)

## The simplest **coding platform**



[Skip to main content](#)

You can see that both the **'while'** and **'do...while'** loops produce the same output. But what happens if you change the \$i so it equals 10 instead of 0?

Have a go at editing the **'while'** and **'do...while'** examples shown in JDoodle so that the variable \$i = 10.



### ☒ 'while' loops

Non-Assessed

What happens when you edit the 'while' and 'do...while' loop examples above so that the \$i variable equals 10? Explain why this happens.

|          |          |          |  |    |                               |    |    |    |  |                |                |  |                    |
|----------|----------|----------|--|----|-------------------------------|----|----|----|--|----------------|----------------|--|--------------------|
| <b>B</b> | <i>I</i> | <u>U</u> |  | :≡ | <sup>1</sup> / <sub>2</sub> ≡ | ≡≡ | ≡≡ | ≡≡ |  | T <sup>2</sup> | T <sub>2</sub> |  | ≡≡                 |
|          |          |          |  |    |                               |    |    |    |  |                |                |  |                    |
|          |          |          |  |    |                               |    |    |    |  |                |                |  | 0 / 500 Word Limit |

Save draft

Submit



[Skip to main content](#)

## 'foreach' loop statements

When it comes to working with arrays and database result sets, **'foreach'** is your friend. Unlike the typical **for** loop, this loop does not expect control variables. The **'foreach'** loop works only on arrays and will iterate over the array variables, looping through each element of that array.

For example, the following **'foreach'** loop statement uses the convention of giving the array a plural name (numbers) and a singular version of the array name (number) to represent the elements of the array.

In this loop iteration, the value of each element in the \$numbers array is assigned to \$number.

```
<?php
$numbers = [1,2,3,4,5];

foreach ($numbers as $number)
{

    echo "Example of foreach: $number\n"; // $number will contain an
    element of the array

};

?>
```

If you require the array key for each element, consider the following example. After the as, use two variables separated by the => operator. The first variable retrieves the key, and the second retrieves the value.

```
<?php

$words = ["one","two","three","four","five"];
```

[Skip to main content](#)

```
foreach ($words as $key => $word) //include the array key
{
    echo "Foreach $key - $word\n"; // $word will contain an element of
    the array, $key its index
};
?>
```

The example below shows the output that is executed for the \$numbers and \$words arrays using the foreach loop.

Can you see that the 'foreach' loop used on the \$words array shows both the key and value pair?



ENTER FULLSCREEN

[Skip to main content](#)

## The simplest **coding platform**



[Skip to main content](#)

Thoughts on this page?

